

COMPOSE: GENERATOR-MATCHED STOCHASTIC REWRITING FOR VALID MOLECULAR GENERATION AND EDITING

Anonymous authors

Paper under double-blind review

ABSTRACT

Flexible molecular generation should support both distribution learning and molecular design: a model must change atom count, composition, bonding, and ring topology while keeping every state exposed to a constraint or property oracle a complete, connected molecule. Existing flexible-size diffusion and flow models typically traverse ambient graph spaces, whereas validity-preserving grammars and constrained processes are largely constructive or confined to fixed molecular fibers. We introduce *Rewrite Generator Matching* (RGM), a generative construction that instantiates Generator Matching on an executable stochastic rewrite system. Stochastic rewriting supplies the state-dependent legal rule-match events and their guarded successors; endpoint-conditioned rewrite programs supply tractable conditional generators; and Generator Matching learns their marginal, time-dependent firing rates. At inference, this yields target-free ancestral CTMC sampling without an endpoint, beam search, candidate bank, or terminal repair. We instantiate RGM as COMPOSE, a flexible-size molecular generator and editor combining general grow, shrink, retype, and bond rewrites with subtree Graft and whole-ring-system moves. Every committed jump preserves connectivity and a declared 2D molecular grammar, while the derived moves coordinate difficult topology changes without reducing the model to fragment assembly. The same rewrite process supports unconditional generation, property targeting, molecular optimization, and source-conditioned editing. Because every committed state is a complete molecule, structural constraints act on the legal event set by construction and property oracles score meaningful intermediates throughout generation, at which COMPOSE maintains all-step validity and connectivity. On ZINC-250k (C/N/O/F) COMPOSE is a sufficiently competitive *de novo* generator, while its distinctive strength is controlled molecular design. It satisfies fixed-scaffold and required/forbidden-substructure constraints *exactly* (100%) where a matched generate-then-filter baseline collapses with constraint hardness (to 12.0% usable on the hardest scaffolds); it spends 100% of its oracle calls on feasible candidates against 39.8% for filtering; and it optimizes QED under an exact scaffold constraint (+0.072 on 12/12 leads, from 0.755 to 0.827) while preserving the scaffold on every lead. On a small exhaustive-state slice of the learned generator, rule-closed conditioning is enforced exactly by the Doob h -transform (endpoint error 1.1×10^{-16}). These results establish executable rewriting as a process substrate for Generator Matching, and pathwise chemical closure as a substrate for exact, oracle-efficient molecular control.

1 INTRODUCTION

Molecular generative modeling asks one model to solve several versions of the same structural problem. In unconditional generation, it must reproduce a molecular distribution. In property targeting and optimization, it must move probability toward desired regions of chemical space. In editing, it must preserve specified structure while revising the rest. All three require freedom to change atom count, composition, bonding, and cyclic topology. They also expose a requirement that endpoint-only metrics can hide: any state consumed by a property oracle, scaffold constraint, or human-interpretable edit should itself be a molecule. Terminal sanitization cannot make an earlier oracle query chemically meaningful.

Graph diffusion and flow models have substantially advanced molecular distribution learning (Jo et al., 2022; Vignac et al., 2023; Jo et al., 2024; Xu et al., 2024; Siraudin et al., 2025; Qin et al., 2025). Their node, edge, and coordinate processes are expressive, but generally evolve in an ambient graph space whose intermediate states need not be connected or valence-admissible. GrIDDD, Edit Flows, and Morph extend this paradigm with insertion, deletion, or substitution, establishing flexible cardinality within learned generative dynamics (Ninniri et al., 2025; Havasi et al., 2025; Franke et al., 2026). A complementary line makes constraints structural: ConStruct restricts graph diffusion, molecular grammars generate through legal productions, CoCoGraph performs validity-preserving swaps, and reaction-based GFlowNets traverse executable synthesis or editing transformations (Madeira et al., 2024; Kajino, 2019; Guo et al., 2022; Ruiz-Botella et al., 2026; Koziarski et al., 2024; Zhu et al., 2026). These are important precedents rather than isolated ingredients to be claimed anew. We are unaware, however, of a Generator-Matched probability path from a directly samplable molecular prior to a data distribution whose learned transitions are simultaneously flexible-size, non-monotone, and closed over complete molecular states.

We obtain such a path by making the interface between two established theories explicit and operational for de novo molecular generation. Stochastic graph rewriting defines qualitative rules, their concrete matches and application conditions, and the CTMC semantics induced by match propensities (Danos et al., 2015; Behr et al., 2021). Generator Matching provides a regression principle for recovering a marginal infinitesimal generator from tractable conditional generators (Holderrieth et al., 2025). The former does not explain how to learn a source-to-data generative transport, and the latter does not choose a molecular state space, executable event vocabulary, or chemistry-closed transition relation. We formulate *Rewrite Generator Matching* (RGM) to supply those objects: endpoint-conditioned executable rewrite programs define conditional generators, and Generator Matching learns their time- and context-dependent marginal firing rates on the rewrite state graph. Thus, established tools produce a new generative construction rather than a validity mask attached to an ambient graph process.

Our molecular realization is COMPOSE—COMpositional Probabilistic Operator Sampling and Editing. A state is a complete, connected graph admitted by a declared 2D atom/bond/charge/implicit-hydrogen grammar. A typed rewrite at a concrete match is a marked jump; application conditions define its state-dependent legal support. Multiple marks can name the same chemical transition because of graph symmetries or alternative executable lowerings, so their rates are aggregated at the canonical molecular successor. This quotient is correctness machinery with precedent in stochastic rewriting and symmetry-aware graph generation, not a separate novelty claim. The important consequence is operational: probability evolves between molecules by executing rules, rather than between unconstrained atom–edge tensors by later interpreting or repairing them.

Training begins with a target-independent, degree-bounded carbon tree $G_0 \sim \pi_0$ and a data molecule $G_1 \sim p_{\text{data}}$. A randomized compiler produces a certified source-to-target rewrite program, and a progress CTMC on that program supplies the conditional rates required by Generator Matching. Marginalizing the target, coupling, program, and progress state yields a generator on observed molecular states, following the conditional-to-marginal and latent-process principles of Generator Matching (Holderrieth et al., 2025; Billera et al., 2026). The compiler is supervision, not a decoder. At inference, a fresh source is transformed by ancestral sampling from the learned CTMC without access to a target, trace, candidate bank, beam search, or terminal repair.

The choice of rewrite paths is also an inductive bias. A purely local compiler can technically correct size, labels, bonds, and topology, yet do so through long or ambiguous programs: a mismatched tree may be pruned and regrown, and a ring system may require tightly coordinated local events. We therefore retain general grow, shrink, retype, and bond rewrites while introducing two verified derived actions. *Subtree Graft* moves a bridge-attached component and reconnects it atomically, revising global topology without exposing a disconnected state. *Whole-ring-system rewrites* coordinate the growth or deletion of supported monocyclic, fused, bridged, spiro, aromatic, and non-aromatic systems. Each derived action has validity-checked primitive lowering and inverse semantics. They compress difficult paths within a universal edit substrate; they do not turn COMPOSE into a fragment vocabulary.

Pathwise closure is especially relevant to molecular design. Hard structural constraints can remove violating matches, conditioning can alter learned rates, and inference-time guidance can reweight only executable molecular successors. Every committed proposal can therefore be scored by a molecular

graph oracle, while non-monotone size and topology changes retain the ability to revise rather than merely decorate a structure. This architectural argument is not itself evidence of better optimization. We consequently stage the evaluation: first establish sufficient unconditional fidelity, coverage, and diversity; then test property targeting, oracle-budgeted optimization, and source-conditioned editing against matched filtering, repair, and guidance baselines. All comparisons account for oracle calls, proposals, samples, and wall-clock cost.

Our contributions are:

- **Rewrite Generator Matching.** We formulate Generator Matching over executable stochastic rewrite systems: certified endpoint-conditioned programs define conditional generators, and neural time-dependent rule-match intensities define a target-free marginal rewrite CTMC. To our knowledge, this is the first instantiation of Generator Matching on a formal, chemistry-closed stochastic graph-rewrite system.
- **A flexible-size, validity-closed molecular realization.** COMPOSE supports non-monotone changes to size, composition, bonds, and ring topology while every committed state remains connected and admitted by the declared 2D molecular grammar. General local rewrites coexist with verified Graft and whole-ring-system path-compression operators.
- **Mechanism- and capability-centered evidence.** We evaluate one rewrite model family across unconditional generation, property targeting, molecular optimization, and editing. All-state audits and matched closure, source, quotient, and micro-versus-macro ablations test which parts of the construction produce the observed behavior.

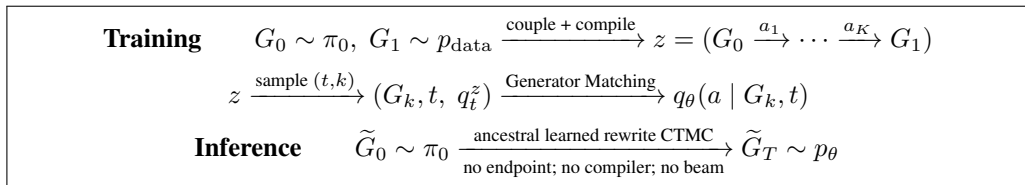


Figure 1: **Method overview.** The endpoint and rewrite program are latent training variables. Generator Matching learns their state-conditional marginal generator; inference repeatedly samples and executes legal marked rewrites.

2 BACKGROUND AND POSITIONING

2.1 GENERATORS AND GENERATOR MATCHING

The infinitesimal generator characterizes a Markov process through its action on test functions. A deterministic flow has $(L_t f)(x) = v_t(x)^\top \nabla f(x)$, a diffusion adds a second-order term, and a pure jump process has

$$(L_t f)(x) = \sum_{y \neq x} Q_t(x, y) [f(y) - f(x)]. \quad (1)$$

Generator Matching is a learning principle across these process classes (Holderrieth et al., 2025): construct tractable endpoint-conditional generators, then regress their conditional expectation given the current state. It does not prescribe a molecular representation, allowed transitions, or source-to-target coupling. RGM contributes those objects for rewrite systems; COMPOSE contributes their chemistry-specific implementation.

2.2 GRAPH DIFFUSION, FLOWS, AND FLEXIBLE SIZE

DiGress and related discrete diffusion models corrupt node and edge categories and learn a denoising process (Vignac et al., 2023; Austin et al., 2021; Campbell et al., 2022); DeFoG formulates graph generation by discrete flow matching (Qin et al., 2025). Their intermediate tensors are discrete graphs but are not restricted to connected valence-admissible molecules. GrIDDD adds monotone node insertion and deletion to graph diffusion (Ninniri et al., 2025). Edit Flows constructs insertion, deletion, and substitution flows for variable-length objects (Havasi et al., 2025). Morph uses unbalanced

optimal transport to change the cardinality of 3D geometric molecular graphs (Franke et al., 2026). DDSBM learns CTMC bridges for graph transformation using independent node/edge modifications and iterative bridge fitting (Kim et al., 2024). These are close in flexibility or mathematical machinery. COMPOSE differs in the support and semantics of its generator: one jump is an executable, guarded chemical rewrite between complete molecules, and de novo inference does not condition on a paired endpoint.

Morph is the closest overlap in its use of Generator Matching, structural priors, and insertion/deletion during molecular generation. Accordingly, neither flexible cardinality, prior-to-data transport, nor Generator Matching by itself is a claimed novelty here. The distinction is that RGM makes a legal rule match—rather than an independently parameterized node, edge, or coordinate update—the marked infinitesimal event, quotients syntactic aliases at the molecular-successor level, and samples entirely within the rewrite-defined chemical state space.

Table 1: Conceptual comparison. “Path support” describes visible generative states, not endpoint validity. “Size edits” means changing cardinality within one trajectory, rather than drawing a size before generation.

Family	Learned infinitesimal object	Path support	Size edits	Typical endpoint information
DiGress / DeFoG	node/edge transition rates	categorical graph tensors	no	none
DDSBM	node/edge bridge rates	categorical graph tensors	no	target distribution / bridge
GrIDDD / Edit Flows	insert, delete, and substitute rates	variable-cardinality objects	yes	none
Morph	unbalanced geometric transport	flexible-size 3D graphs	yes	none or condition
CoCoGraph	valid double-edge-swap process	fixed formula/degree fiber	no	formula and degree sequence
Molecular grammars	derivation policy	grammar-valid partial derivations	yes	none
Stochastic rewriting	rule-match propensity	rule-defined graph states	rule-dependent	none
COMPOSE	legal rewrite-match rate	complete grammar-valid connected molecules	yes	none at inference

2.3 CONSTRAINTS, GRAMMARS, AND STOCHASTIC REWRITING

ConStruct projects a discrete graph diffusion process onto selected structural constraints (Madeira et al., 2024); CoCoGraph uses connectivity-preserving double-edge swaps inside fixed composition/degree fibers (Ruiz-Botella et al., 2026). Molecular hypergraph grammars can make validity structural and improve data efficiency (Kajino, 2019; Guo et al., 2022); fragment and junction-tree models generate through a learned or predefined motif vocabulary (Jin et al., 2018). Stochastic graph rewriting provides the closest operational precedent: qualitative rules define matches and application conditions, while contextual propensities induce a CTMC (Danos et al., 2015; Behr et al., 2021). COMPOSE imports this rule/rate separation, but replaces fixed or thermodynamic propensities with a time-inhomogeneous neural generator learned from endpoint-conditioned programs. Its primitive operators, deletion and correction semantics, and source-independent sampling also distinguish it from a fragment assembly ontology.

3 REWRITE GENERATOR MATCHING

3.1 MOLECULAR STATES, LEGAL MATCH FIBERS, AND QUOTIENT RATES

Let a padded heavy-atom graph be $G = (V, E, \ell, b)$, where ℓ_v contains element, formal charge, and implicit-hydrogen state and b_{uv} is a discrete bond class. Given chemistry grammar Γ and atom cap $N_{\max}$, define

$$\mathcal{M}_\Gamma = \{G : 1 \leq |V| \leq N_{\max}, G \text{ is connected and valence-admissible under } \Gamma\}. \quad (2)$$

This is a representation-level domain: it does not assert synthesizability, low strain, or 3D feasibility.

A rule $r \in \mathcal{R}$ has a left pattern, a right pattern, and application conditions. A concrete mark $a = (r, m, \xi)$ consists of a match $m \in \text{Match}_r(G)$ and a typed payload ξ (for example a new atom state, bond order, or ring program). Its executor T_a is partial. The legal marked-action fiber is

$$\mathcal{A}(G) = \{(r, m, \xi) : m \in \text{Match}_r(G), \text{Legal}(G, r, m, \xi) = 1\}, \quad \mathsf{T}_a G \in \mathcal{M}_\Gamma \quad \forall a \in \mathcal{A}(G). \quad (3)$$

A neural marked rate $q_\theta(a \mid G, t) \geq 0$ defines

$$(L_{\theta, t} f)(G) = \sum_{a \in \mathcal{A}(G)} q_\theta(a \mid G, t) [f(\mathsf{T}_a G) - f(G)]. \quad (4)$$

The marked representation is not unique: graph automorphisms, Kekulé alternatives, or distinct macro lowerings can name the same successor. Let $a \sim_G a'$ iff $\mathsf{T}_a G$ and $\mathsf{T}_{a'} G$ are the same canonical chemical state. The quotient successor fiber and its rate are

$$\mathcal{S}(G) = \{H \in \mathcal{M}_\Gamma : \exists a \in \mathcal{A}(G), H \simeq \mathsf{T}_a G\}, \quad (5)$$

$$Q_{\theta, t}(G, H) = \sum_{a \in \mathcal{A}(G) : \mathsf{T}_a G \simeq H} q_\theta(a \mid G, t). \quad (6)$$

Equation (6) is essential: a model may parameterize named matches, but probability evolves on chemical states through the sum of all aliases.

3.2 LATENT REWRITE PROGRAMS AS CONDITIONAL PROBABILITY PATHS

Sample a source–target pair from a coupling γ with marginals π_0 and p_{data} . A randomized compiler samples a certified program

$$Z = (G_0, a_1, G_1^Z, \dots, a_K, G_K^Z), \quad G_{k+1}^Z = \mathsf{T}_{a_{k+1}} G_k^Z \in \mathcal{M}_\Gamma, \text{quad} G_K^Z \simeq G_1. \quad (7)$$

The superscript distinguishes program states from the paired endpoint. The compiler distribution is part of the training coupling; multiple programs or source draws may be sampled for one endpoint.

Let $\alpha : [0, 1] \rightarrow [0, 1]$ be increasing with $\alpha(0) = 0$ and $\alpha(1) = 1$. Conditional on a length- K program, define progress $K_t \sim \text{Binomial}(K, \alpha(t))$ and $X_t = G_{K_t}^Z$. The pure-birth realization fires only the next instruction at latent progress $k < K$, with marked rate

$$r_t^Z(a \mid k) = \mathbf{1}\{a = a_{k+1}\} (K - k) \frac{\alpha'(t)}{1 - \alpha(t)}. \quad (8)$$

For $\alpha(t) = t$, operational time $s = -\log(1 - t)$ removes the terminal singularity and gives rate $K - k$. Sampling k directly from its binomial marginal avoids simulating the teacher chain during training. If a program revisits the same molecular state, k remains part of the latent variable; the learned rate conditions only on X_t and therefore averages over those occurrences.

The conditional successor rate is

$$R_t^Z(H \mid X_t, k) = \sum_{a : \mathsf{T}_a X_t \simeq H} r_t^Z(a \mid k), \quad (9)$$

and the marginal generator to be learned is

$$Q_t(G, H) = \mathbb{E}[R_t^Z(H \mid X_t, K_t) \mid X_t = G]. \quad (10)$$

This conditional expectation is the step inherited from general Generator Matching. The rewrite state space, program construction, legal fibers, and successor quotient are specific to RGM.

Table 2: Rewrite families. A derived rule is admitted only if its macro transaction and verified lowering satisfy the same executor contract.

Family	Semantic effect	Generative role
Grow / shrink	attach a typed atom; delete a removable atom	variable size with no disconnected state
Atom restate	jointly change element, charge, and implicit H	composition and local-valence correction
Bond edit	insert/delete a non-bridge bond or change bond class	ring repair and unsaturation
Subtree Graft	cut one bridge and reattach the detached component atomically	global topology revision without atom churn
Ring ear	insert/delete a path between anchors, including same-anchor cycles	coordinated cyclic, fused, bridged, and spiro topology
Ring restate	verified joint electronic restatement on a fixed cyclic system	aromatic and heteroatom coordination

3.3 POISSON-BREGMAN GENERATOR MATCHING OBJECTIVE

For $\phi(u) = u \log u - u$, the Bregman divergence between nonnegative rates gives, up to target-only constants,

$$\mathcal{L}_{\text{RGM}}(\theta) = \mathbb{E}_{t,Z,K_t} \left[\sum_{H \in \mathcal{S}(X_t)} Q_{\theta,t}(X_t, H) - R_t^Z(H | X_t, K_t) \log Q_{\theta,t}(X_t, H) \right]. \quad (11)$$

The first term calibrates total firing mass; the second rewards the conditional teacher successor. Unlike a classification loss over the observed action, it is proper for rates. Training can oversample rare rule-family progress positions using a mixture proposal $\tilde{p}(k | t, Z)$ and weight by $p(k | t, Z)/\tilde{p}(k | t, Z)$; this changes variance, not the population objective.

4 COMPOSE: A MOLECULAR REWRITE PROCESS

4.1 PRIMITIVE AND DERIVED OPERATOR SYSTEMS

The micro language contains leaf insertion/deletion, atom restatement, bond insertion/deletion, and bond reordering. It is expressive but can be statistically inefficient: a full ring requires coordinated local decisions, while tree topology may be changed only after destructive atom churn. Derived rules compress recurring valid programs into one semantic mark. They do not create a fragment vocabulary: payloads factorize over topology, anchors, atom/bond states, and span, and the micro rules retain support for uncommon structures.

Subtree Graft. Choose a bridge (u, v) , conceptually remove it, choose endpoints in the two induced components, and add a new cross-component bond in the same atomic transaction. The old bridge cannot be exposed as a visible disconnected state. Graft preserves atom count and the full detached component, providing direct supervision for correction of random source-tree wiring.

Coordinated rings. Root-cycle and attached-cycle rules introduce a closed cyclic block; an open-ear rule inserts a path between two existing anchors; a same-anchor ear introduces a spiro petal; and a ring-system restatement coordinates common bond-order/heteroatom patterns. Open-ear decomposition supplies a compact topological substrate for monocyclic, fused, and bridged biconnected blocks, while same-anchor composition covers spiro articulation. Aromaticity is represented by executable bond/atom states, not an unconstrained post-hoc label. All ring sizes remain structurally possible subject to N_{max} and grammar guards; corpus frequencies belong in learned rates, not hard support.

4.2 A DIRECTLY SAMPLABLE CARBON-TREE PRIOR

The principal de novo prior first samples a heavy-atom count $N \sim \mu$, then a degree-bounded tree by a capped Prüfer construction, and realizes it as a neutral, single-bonded carbon graph. Thus $G_0 \sim \pi_0$

Ancestral rewrite sampling. Sample $G \sim \pi_0$ and set operational time $s = 0$. While $s < S$ and the event budget is not exhausted: (i) predict total legal hazard and masked marked probabilities; (ii) draw an exponential waiting time; (iii) if it crosses a frozen-rate boundary, advance to that boundary; otherwise sample one legal marked rewrite, execute $G \leftarrow T_a G$, and advance by the waiting time. Return the current complete molecule.

Figure 2: Target-free inference. A returned state is meaningful at every stopping time because the process never leaves $\mathcal{M}_\Gamma$.

is connected, valence-admissible, and sampled without a target. The random tree is structured noise, not a retained scaffold; every atom may be retyped, moved by Graft, deleted, or replaced.

Coupling source and target sizes can shorten teacher paths without leaking an endpoint at inference. In the size-matched experiment, $N_0 = N_1$ and $N_1 \sim \mu$; hence the unconditional source-size marginal is still μ . Our first flexible-size compiler instead draws $N_0 \sim \mu$ independently of N_1 , which exactly matches the ancestral prior and directly supervises both grow and shrink, but can produce longer programs. A lower-variance alternative is a size-jitter kernel $J(N_0 | N_1)$ satisfying stationarity $\sum_n \mu(n) J(m | n) = \mu(m)$; a reversible Metropolis kernel on atom counts is one construction. The source wiring remains independent given N_0 . We separately evaluate a null prior, an independently sized tree, a size-matched tree, and stationary size-jitter so that topology and cardinality effects are not conflated.

4.3 CERTIFIED PROGRAM COMPILATION

For equal-sized source trees, the compiler aligns temporary slots, selects a target spanning tree that maximizes overlap with the source, and uses Graft edge exchanges to reach that scaffold. It then restates atom states and retained bond classes and installs omitted target chords through ring-ear rewrites. Size-mismatched programs first delete or insert atoms under connectivity guards. Every emitted action is executed immediately, every intermediate is validated, and the final canonical graph must equal the target.

This offline planning problem is distinct from sampling. The compiler may use deterministic tie-breaking, bounded search, or a constructive fallback because it sees both endpoints. The learned sampler sees none of these. Calling inference “ancestral” means that each next event is sampled from the current learned hazard and then becomes part of the conditioning state; it does not mean fixed-order autoregression.

4.4 FACTORIZED NEURAL RATES AND EXACT SUPPORT MASKS

Enumerating and encoding every successor separately is prohibitive. COMPOSE encodes the molecular graph once and factorizes a marked rate as

$$q_\theta(a | G, t) = \lambda_\theta(G, t) p_\theta(r | G, t) p_\theta(m, \xi | r, G, t), \quad (12)$$

where both categorical factors are normalized after analytic legality masks. A message-passing encoder provides graph, node, pair, and ring-context features. The total-hazard head learns how fast events occur; family and operand heads learn which legal event fires. Only the sampled mark is instantiated and passed to the full executor. An exhaustive canonical-successor enumerator remains a small-system oracle for mask equivalence, alias aggregation, and sampling tests.

4.5 TARGET-FREE ANCESTRAL CTMC SAMPLING

Starting at $G \sim \pi_0$, the sampler evaluates $\lambda_\theta(G, t)$, draws a waiting time, samples a masked family and payload, commits the rewrite, and recomputes rates from the successor. Rates are piecewise frozen between time-grid boundaries. There is no target, path compiler, candidate bank, beam, or terminal repair. Algorithm 2 is detailed in Appendix E.3.

5 THEORETICAL PROPERTIES

We separate support guarantees, which follow from the rewrite system, from distributional correctness, which additionally requires exact Generator Matching.

Assumption 1 (Finite closed rewrite system). $N_{\max}$ and all type vocabularies are finite; every committed action satisfies (3); and the total rate is locally bounded on the finite time horizon.

Assumption 2 (Conditional path consistency). The program sampler has endpoint marginals $G_0 \sim \pi_0$ and $G_K^Z \sim p_{\text{data}}$, and every sampled program satisfies (7).

Proposition 3 (Pathwise molecular closure). *Under assumption 1, if $G_0 \in \mathcal{M}_\Gamma$, every state committed by the teacher or learned CTMC lies in $\mathcal{M}_\Gamma$ almost surely.*

Proposition 4 (Connected topological reachability). *Ignoring chemical labels, any two finite connected simple graphs can be connected by a finite sequence of leaf insertions/deletions and non-bridge edge insertions/deletions, with every intermediate connected. For equal-order trees, subtree Grafts suffice using at most $(n - 1) - |E(T_0) \cap E(T_1)|$ edge exchanges under a fixed vertex alignment.*

Remark 5. The first statement is topological, not a universal chemistry theorem. In COMPOSE, every proposed step must also satisfy the valence grammar. Dataset-pair reachability is therefore certified constructively by compiler replay rather than inferred from Proposition 4 alone.

Proposition 6 (Population RGM minimizer). *For every occupied (G, t) and successor H , the unrestricted nonnegative-rate minimizer of (11) is the marginal quotient rate $Q_t(G, H)$ in (10).*

Corollary 7 (Endpoint law under an exact model). *Under assumptions 1 and 2, if $Q_{\theta, t} = Q_t$ on occupied states and the learned process starts at π_0 , its time marginals match the program-mixture marginals; in particular its terminal law is p_{data} .*

The proofs are in Appendix D. These claims do not imply that a finite network attains the population minimizer, that teacher programs are optimal, or that 2D validity implies synthetic accessibility.

6 EXPERIMENTS

6.1 QUESTIONS, HYPOTHESES, AND PROTOCOL

The evaluation stages sufficiency before control: we first establish that COMPOSE is a credible *de novo* generator, then test the paper’s empirical thesis—exact, oracle-efficient structural control—against matched filtering baselines. It is organized around falsifiable questions.

- Q1. **De novo sufficiency:** is COMPOSE a valid, diverse, non-memorizing generator with faithful marginals and coverage—a credible base, not a distribution-learning leaderboard entry?
- Q2. **Exact structural control:** does COMPOSE satisfy fixed-scaffold and required/forbidden-substructure constraints by construction, and how does a matched generate-then-filter baseline degrade as the constraint hardens?
- Q3. **Oracle efficiency:** because feasibility is enforced before scoring, what fraction of oracle calls does COMPOSE spend on feasible candidates relative to filtering?
- Q4. **Constrained optimization and anytime use:** does COMPOSE improve a property (QED) while holding a scaffold exactly, with every intermediate a usable, improving candidate?
- Q5. **Exactness anchor:** on a tractable exhaustive-state slice, does the Doob h -transform of the learned generator enforce rule-closed conditioning exactly?
- Q6. **Support and mechanism:** do all accepted intermediates remain valid and connected, and do Graft, coordinated rings, and non-monotone grow/shrink/retype behave as designed (no executor/mask discrepancy, no delete-to-one collapse)?

Data and preprocessing. The reported evaluation uses ZINC-250k restricted to C/N/O/F with charged N^+/O^- , matching the preprocessing of our primary flexible-size comparator GrIDDD (Ninniri et al., 2025) so the comparison is on equal chemical footing. We canonicalize with a frozen RDKit version, declare the atom/charge/bond vocabulary and $N_{\max} = 40$, and hash the accepted split. The conditional leads are the CNOF subset of the Jin et al. constrained-optimization test set

(QED $\in [0.7, 0.8]$). All exclusions and typed-ring catalog coverage are reported by split; validation and test molecules never construct proposal vocabularies or source-size statistics. A GuacaMol distribution-learning table under the standard protocol is deferred to the final version and is not required for the control claims.

Metrics. *De novo sufficiency* is reported by validity, uniqueness, novelty, physicochemical descriptor-distribution distances (logP, QED, molecular weight, TPSA, and ring descriptors, in the MOSES/GuacaMol convention), internal diversity, and train-support coverage; FCD is deferred to the final GuacaMol distribution-learning table and is not reported here (we do not stage an FCD arms race). *Control* is reported by the exact constraint-satisfaction rate, the generate-then-filter usable fraction stratified by constraint hardness, usable-oracle efficiency (feasible-scored over all oracle calls), constrained property gain at fixed scaffold, and anytime best-so-far reward versus oracle calls. Path metrics count every committed state, zero-hazard stalls, event-budget exhaustion, per-family events, source-edge survival, and invalid attempts; because masked actions should be impossible, any all-step failure is reported as a software violation rather than averaged away.

Baselines and fairness. We compare with DiGress, DeFoG, GrIDDD, Morph, CoCoGraph, and grammar/fragment systems only where an official artifact can be evaluated under compatible chemistry and splits. Rerun and paper-reported values are labeled separately. DDSBM is evaluated as a graph-transformation neighbor, not presented as an unconditional *de novo* baseline. Parameter count, preprocessing, accelerator hours, sampling evaluations, and sanitization policy are reported.

6.2 CONTROLLED MOLECULAR DESIGN (PRIMARY RESULTS)

All results here are frozen committed artifacts on ZINC-250k CNOF with the Lineage-B base. The generate-then-filter baseline draws proposals from the *same* base model and applies each constraint as a post-hoc filter, isolating the value of enforcing constraints *in the legal fiber* rather than after scoring.

The constraints we study—a fixed scaffold, a required or forbidden motif—are the ones molecular design actually uses, yet they are *subgraph-existence* conditions rather than global graph properties. This is precisely why they are not “easy”: a diffusion in an ambient graph space cannot mask them mid-process, because whether a trajectory will *end* containing a specified labeled substructure is undecidable from an incomplete, possibly disconnected intermediate, and prior learned constrained generators instead enforce graph-theoretic (planarity, acyclicity, degree) or formula/degree constraints. Because every committed COMPOSE state is a complete molecule, the same conditions are checkable—and therefore enforceable in the legal fiber—at every step by construction. It is the process structure, not a choice of easy constraints, that makes the design-relevant constraints exact for COMPOSE and out of reach for the dominant paradigm.

No prior *learned* constrained generator can represent these constraints, which is why generate-then-filter is the matched control rather than a straw man. ConStruct’s projector is provably confined to edge-deletion-invariant topological properties—planarity, acyclicity, lobster structure, bounded degree—whereas a required Murcko scaffold or required SMARTS motif is edge-*insertion*-invariant, the opposite class, and cannot be expressed (Madeira et al., 2024). PRODIGY projects onto aggregate count and threshold sets (edge, triangle, and atom counts, degree, valency, molecular weight) and provides no operator for a specific labeled subgraph (Sharma et al., 2024). CoCoGraph fixes the molecular formula, valence/degree sequence, and connectivity through double-edge swaps that deliberately rewire which atoms bond, so a designated scaffold is not representable (Ruiz-Botella et al., 2026). Generate-then-filter—drawing from the same base model and rejecting violations post hoc—is thus the only baseline that can *target* these labeled-subgraph constraints at all; it satisfies them only by chance, and that chance vanishes as the constraint hardens (Figure 3).

Exact rule-closed constraints (Q2). Fixing a Murcko scaffold, or a required/forbidden SMARTS motif, removes violating marks from $\mathcal{A}(G)$, so COMPOSE satisfies the constraint by construction: **100%** scaffold satisfaction and **100%** required/forbidden-substructure satisfaction across all leads. The matched generate-then-filter baseline does not degrade gracefully: its usable fraction collapses monotonically with scaffold size (Table 3; Figure 3), from 95.7% on small scaffolds (≤ 8 atoms) to 12.0% on the hardest (≥ 23 atoms). On the 32/47 leads whose scaffolds have ≥ 16 atoms the baseline averages $\approx 19\%$ usable (≈ 3.8 of 20 candidates) versus 20/20 for COMPOSE.

Table 3: Fixed-scaffold satisfaction by scaffold size (47 leads, ZINC-CNOF). COMPOSE enforces the scaffold in the legal fiber; generate-then-filter applies it as a post-hoc filter. Entries are the fraction of scored candidates that satisfy the constraint—equivalently, the usable-oracle efficiency of that method.

Scaffold heavy atoms	≤ 8	9–15	16–22	≥ 23	overall
Generate-then-filter	95.7%	79.9%	29.4%	12.0%	39.8%
COMPOSE (in-fiber)	100%	100%	100%	100%	100%

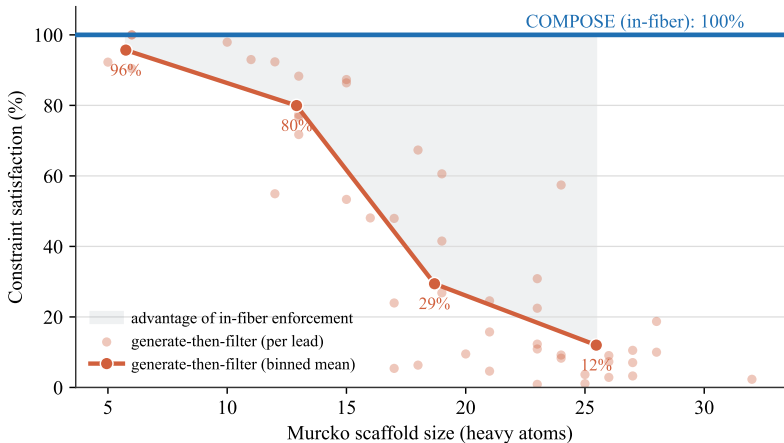


Figure 3: **Exact scaffold satisfaction versus generate-then-filter collapse.** Enforcing a fixed Murcko scaffold in the legal fiber holds COMPOSE at 100% satisfaction for all leads (blue), whereas drawing the same base model’s proposals and filtering post hoc collapses monotonically with scaffold size (terracotta: per-lead points, with binned means annotated). The shaded region is the widening advantage of in-fiber enforcement. Data: 47 ZINC-CNOF leads.

Usable-oracle efficiency (Q3). Because feasibility is checked before scoring, every oracle call COMPOSE makes lands on a feasible candidate—**100%** usable-oracle efficiency by construction. Generate-then-filter must score every candidate before discarding, so its efficiency equals its constraint-satisfaction fraction: 39.8% overall, wasting up to 88% of the oracle budget on the hardest scaffolds. This is a structural, not tuned, sample-efficiency advantage on the axis oracle-budgeted optimization is measured.

Constrained optimization and anytime behaviour (Q4). Value-guided sequential Monte Carlo over the rewrite CTMC, with the scaffold enforced in the fiber, improves QED on 12/12 leads (mean $0.755 \rightarrow 0.827$, $+0.072$) while preserving the scaffold on every lead at **100%** oracle efficiency. Because every committed state is a usable molecule, the process is anytime (Figure 4): most of the gain is realized early—the median lead reaches $\approx 90\%$ of its final gain within 13 oracle calls—while a minority improve through a later jump, and sampling can be stopped, inspected, or resumed at any intermediate. We deliberately do *not* report an unconstrained QED success-rate race: our oracle-guided SMC is oracle-hungry and is not a matched comparison to oracle-light conditional models (Section 7); QED here is the objective inside a constrained task, not a leaderboard claim.

Exactness anchor (Q5). On an exhaustive 118-state size-capped slice of the production legal-event CTMC, the Doob h -transform of the learned generator enforces a rule-closed target-set condition exactly: the tilted one-step kernels reproduce the exact Bayes conditional endpoint to machine precision (ℓ_∞ error 1.1×10^{-16}), the tilted generator Q^h is a valid generator (off-diagonal negativity 0.0, row-sum error 2×10^{-13}), and generator-level integration converges first-order to the exact conditional, across conditioning predicates. This is the one place we claim exactness: in the full state space the harmonic function is intractable, so property conditioning uses the approximate value-guided SMC above, and Corollary 7 gives the endpoint law only under an exact model.

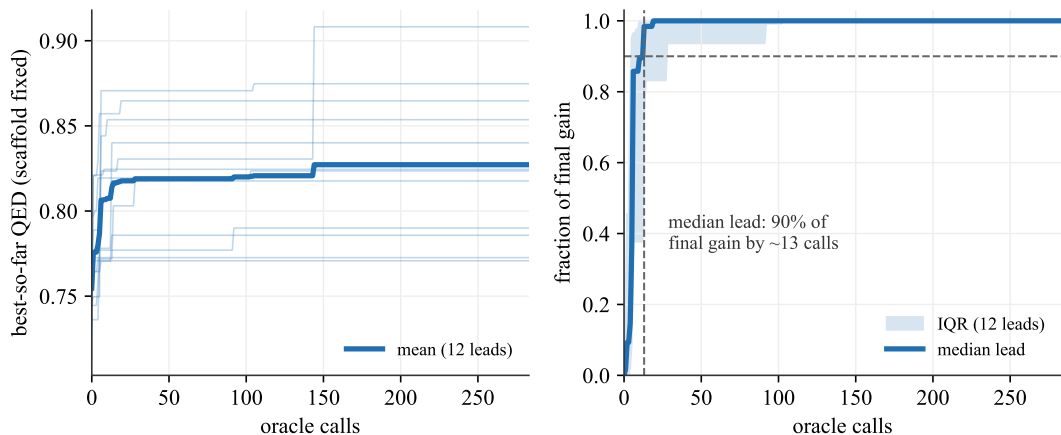


Figure 4: **Anytime constrained optimization.** With the Murcko scaffold enforced in the fiber, best-so-far feasible QED climbs quickly and plateaus, so every intermediate is a usable, improving candidate (left: 12 per-lead traces with bold mean). Right: fraction of each lead’s final gain versus oracle calls (median with interquartile band); the median lead reaches 90% of its final gain within ~ 13 oracle calls, with a minority improving via a later jump.

Table 4: De novo sufficiency on ZINC-CNOF. This is the sufficiency gate for Section 6.2, not a distribution-learning leaderboard; FCD is deferred to the final GuacaMol table and not reported here. COMPOSE validity is 100% *by construction* (Proposition 3); its remaining entries are from the frozen sufficiency run, and baseline cells are to be run or cited. Desc. W_1 is the mean physicochemical descriptor Wasserstein distance (logP, QED, MW, TPSA, rings).

Method	Valid $\uparrow$	Unique $\uparrow$	Novel $\uparrow$	Desc. W_1 $\downarrow$	Int. div. $\uparrow$	Train cov. $\uparrow$
DiGress	XXX	XXX	XXX	XXX	XXX	XXX
DeFoG	XXX	XXX	XXX	XXX	XXX	XXX
GrIDDD	XXX	XXX	XXX	XXX	XXX	XXX
Morph	XXX	XXX	XXX	XXX	XXX	XXX
COMPOSE	100%	XXX	XXX	XXX	XXX	XXX

6.3 DE NOVO SUFFICIENCY (CONFIRMATORY EVALUATION PENDING)

The de novo tables below are the sufficiency gate of Section 6.2, not a distribution-learning leaderboard; developmental smoke values (Section 6.5) are near-100% valid/unique/novel with element-distribution TV ≈ 0.03 , but final entries remain **XXX** until a frozen multi-seed evaluation at $\geq 10,000$ samples is complete.

6.4 ABLATIONS AND PROCESS DIAGNOSTICS

Reachability alone does not establish learned capability. We therefore report event fractions for Graft (**XXX**), grow (**XXX**), shrink (**XXX**), retype (**XXX**), and ring events (**XXX**), stratified by model time and source/endpoint size. We additionally measure whether source edges survive, whether paths visit the one-atom state, and whether samples have the covariance collapse that aggregate FCD can obscure.

6.5 DEVELOPMENTAL EVIDENCE IS NOT A BENCHMARK RESULT

Current engineering evidence establishes executability, not final quality. In a developmental size-matched Graft run, the compiler produced 99,738 supported programs for 49,869 of 50,000 targets; a 3,000-update checkpoint reduced held-out RGM loss from 84.09 to 28.14 and reached 90.2% teacher family accuracy. Separately, an early 1,000-sample gate produced 100% valid/connected accepted rollouts on its restricted chemistry. These values diagnose the implementation and motivate the

Table 5: Path and structural fidelity. All-step metrics audit every accepted intermediate, not only endpoints. Values are pending confirmatory evaluation.

Variant	Step valid	Step conn.	Size TV	Cycle TV	Ring TV	Budget	Events
Null prior	XXX	XXX	XXX	XXX	XXX	XXX	XXX
Independent tree	XXX	XXX	XXX	XXX	XXX	XXX	XXX
Size-matched Graft	XXX	XXX	XXX	XXX	XXX	XXX	XXX
Flexible-size Graft	100%	100%	XXX	XXX	XXX	XXX	XXX

Table 6: Matched ablations isolating which construction choices produce the observed behaviour. Path length is measured on compiled teachers; the distribution distance (mean physicochemical descriptor W_1) and ring-size TV are measured from target-free learned samples. All values remain XXX pending the frozen study.

Variant	Desc. W_1 ↓	Mean teacher length ↓	Ring-size TV ↓
Null source	XXX	XXX	XXX
Primitive tree (delete/regrow)	XXX	XXX	XXX
Tree + Graft	XXX	XXX	XXX
No coordinated ring macros	XXX	XXX	XXX
No successor alias aggregation	XXX	XXX	XXX
No importance-correct stratification	XXX	XXX	XXX

confirmatory experiment. They are not directly comparable to full GuacaMol baselines, are not inserted into Tables 4–6, and will be replaced by frozen multi-seed results.

7 DISCUSSION AND LIMITATIONS

The intended contribution is a new division of labor. Rewriting defines a valid state graph and executable event semantics; Generator Matching learns how probability should flow on that graph. The payoff is where intermediate molecules are consumed: online guidance scores every state, scaffold and substructure constraints remove actions from the legal fiber, and source-conditioned editing reuses the same correction operators. Section 6.2 reports these as primary results—exact rule-closed constraints, usable-oracle efficiency, an exactness anchor, and constrained optimization. What we do *not* claim is a competitive *unconstrained* property-optimization result: our oracle-guided SMC is oracle-hungry (≈ 560 calls) and is not a matched comparison to oracle-light conditional models such as GrIDDD (≈ 20 calls); a learned oracle-light route (classifier-free conditioning, reward fine-tuning) remains weak and is left to future work. QED optimization is therefore reported only inside constrained tasks, never as a standalone success-rate leaderboard.

The current state is a 2D heavy-atom graph with implicit hydrogens. Stereochemistry, conformers, radicals, metals, and multi-center bonding require extensions. Declared valence and RDKit sanitization are not guarantees of synthesizability. A finite typed ring catalog can under-cover rare teacher programs; the factorized ring decoder must be evaluated against that failure mode. The compiler introduces inductive bias and may concentrate supervision on one of many valid paths. A carbon-tree prior is useful only if it improves learned transport rather than merely shortening teachers. Finally, hard support guarantees do not imply low FCD, likelihood optimality, rapid mixing, or chemical usefulness.

8 CONCLUSION

COMPOSE turns stochastic molecular graph rewriting from a hand-designed simulator or grammar into the transition substrate of a learned generative process. Endpoint-conditioned valid programs provide exact jump-rate targets; Generator Matching removes the endpoint and program; guarded execution preserves the declared chemical state space at every committed event. Subtree Grafts and coordinated ring rules make the universal operator story practical without replacing it with fragment assembly. The committed results make pathwise chemical closure a substrate for *exact*, *oracle*-

efficient molecular control: rule-closed constraints satisfied by construction where generate-then-filter collapses, all oracle calls spent on feasible candidates, an exactness anchor verified to machine precision, and property optimization under an exactly held scaffold. The remaining confirmatory tests, marked XXX, establish de novo sufficiency at frozen multi-seed scale.

REFERENCES

- Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured denoising diffusion models in discrete state-spaces. In *Advances in Neural Information Processing Systems*, volume 34, pp. 17981–17993, 2021.
- Nicolas Behr, Jean Krivine, Jakob L. Andersen, and Daniel Merkle. Rewriting theory for the life sciences: A unifying theory of CTMC semantics. *arXiv preprint arXiv:2106.02573*, 2021. URL <https://arxiv.org/abs/2106.02573>.
- Lukas Billera, Hedwig Nora Nordlinder, and Ben Murrell. Latent process generator matching. *arXiv preprint arXiv:2605.20547*, 2026. URL <https://arxiv.org/abs/2605.20547>.
- Andrew Campbell, Joe Benton, Valentin De Bortoli, Tom Rainforth, George Deligiannidis, and Arnaud Doucet. A continuous time framework for discrete denoising models. In *Advances in Neural Information Processing Systems*, volume 35, pp. 28266–28279, 2022.
- Vincent Danos, Russell Harmer, and Ricardo Honorato-Zimmer. Thermodynamic graph-rewriting. *arXiv preprint arXiv:1503.06022*, 2015. URL <https://arxiv.org/abs/1503.06022>.
- Malte Franke, Stefan P. Schmid, Žarko Ivković, Kjell Jorner, and Andreas Krause. Generative molecular morphing for flexible-size design via unbalanced optimal transport. *arXiv preprint arXiv:2606.07239*, 2026.
- Minghao Guo, Veronika Thost, Beichen Li, Payel Das, Jie Chen, and Wojciech Matusik. Data-efficient graph grammar learning for molecular generation. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=l4IHwGq6a>.
- Marton Havasi, Brian Karrer, Itai Gat, and Ricky T. Q. Chen. Edit flows: Flow matching with edit operations. *arXiv preprint arXiv:2506.09018*, 2025.
- Peter Holderrieth, Marton Havasi, Jason Yim, Neta Shaul, Itai Gat, Tommi Jaakkola, Brian Karrer, Ricky T. Q. Chen, and Yaron Lipman. Generator matching: Generative modeling with arbitrary markov processes. In *International Conference on Learning Representations*, 2025.
- Wengong Jin, Regina Barzilay, and Tommi Jaakkola. Junction tree variational autoencoder for molecular graph generation. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pp. 2323–2332, 2018.
- Jaehyeong Jo, Seul Lee, and Sung Ju Hwang. Score-based generative modeling of graphs via the system of stochastic differential equations. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pp. 10362–10383, 2022.
- Jaehyeong Jo, Dongki Kim, and Sung Ju Hwang. Graph generation with diffusion mixture. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, 2024.
- Hiroshi Kajino. Molecular hypergraph grammar with its application to molecular optimization. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 3183–3191. PMLR, 2019. URL <https://proceedings.mlr.press/v97/kajinol9a.html>.
- Jun Hyeong Kim, Seonghwan Kim, Seokhyun Moon, Hyeongwoo Kim, Jeheon Woo, and Woo Youn Kim. Discrete diffusion schrödinger bridge matching for graph transformation. *arXiv preprint arXiv:2410.01500*, 2024.

- Michał Koziarski, Andrei Rekesh, Dmytro Shevchuk, Almer van der Sloot, Piotr Gaiński, Yoshua Bengio, Cheng-Hao Liu, Mike Tyers, and Robert A. Batey. RGFN: Synthesizable molecular generation using GFlowNets. *arXiv preprint arXiv:2406.08506*, 2024.
- Manuel Madeira, Clément Vignac, Dorina Thanou, and Pascal Frossard. Generative modelling of structurally constrained graphs. In *Advances in Neural Information Processing Systems*, 2024.
- Matteo Ninniri, Marco Podda, and Davide Bacciu. Graph diffusion that can insert and delete. *arXiv preprint arXiv:2506.15725*, 2025.
- Yiming Qin, Manuel Madeira, Dorina Thanou, and Pascal Frossard. Defog: Discrete flow matching for graph generation. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, 2025.
- Manuel Ruiz-Botella, Marta Sales-Pardo, and Roger Guimerà. A collaborative constrained graph diffusion model for the generation of realistic synthetic molecules. *Nature Machine Intelligence*, 2026. doi: 10.1038/s42256-026-01229-5.
- Kartik Sharma, Srijan Kumar, and Rakshit Trivedi. Diffuse, sample, project: Plug-and-play controllable graph generation. In *International Conference on Machine Learning (ICML)*, 2024. PMLR v235; verify final title/pages before submission.
- Antoine Siraudin, Fragkiskos D. Malliaros, and Christopher Morris. Cometh: A continuous-time discrete-state graph diffusion model. *Transactions on Machine Learning Research*, 2025.
- Clément Vignac, Igor Krawczuk, Antoine Siraudin, Bohan Wang, Volkan Cevher, and Pascal Frossard. Digress: Discrete denoising diffusion for graph generation. In *International Conference on Learning Representations*, 2023.
- Zhe Xu, Ruizhong Qiu, Yuzhong Chen, Huiyuan Chen, Xiran Fan, Menghai Pan, Zhichen Zeng, Mahashweta Das, and Hanghang Tong. Discrete-state continuous-time diffusion for graph generation. In *Advances in Neural Information Processing Systems*, 2024.
- Yuchen Zhu, Donghai Zhao, Yangyang Zhang, Yitong Li, Xiaorui Wang, Shuwang Li, Yue Kong, Beichen Zhang, Ricki Chen, Chang Liu, Xingcai Zhang, Tingjun Hou, and Chang-Yu Hsieh. Designing the haystack: Programmable chemical space for generative molecular discovery. *arXiv preprint arXiv:2603.00614*, 2026.

A OPERATOR SEMANTICS AND CONTRACTS

For each r , $\text{Match}_r(G)$ is the finite set of pattern monomorphisms satisfying structural application conditions. The payload ξ contains new atom/bond states or a macro program. Rule, match, and payload define a partial map $T_{r,m,\xi} : \mathcal{M}_\Gamma \rightarrow \mathcal{M}_\Gamma$. The implementation follows a propose–validate–commit discipline: construct a candidate copy, update implicit hydrogens and bond states jointly, run local valence and global sanitization/connectivity checks, then commit or expose no transition.

Why macros remain rewrite events. A macro has one chemically interpretable successor and one learned rate. Its lowering is a proof-carrying implementation artifact: each micro state must also lie in $\mathcal{M}_\Gamma$, but inference exposes the macro as one marked CTMC event. This avoids treating arbitrary edit bundles as simultaneous jumps. Unrelated micro events may conflict on valence, fail to commute, and create an exponential bundle space; only verified compound semantics are promoted.

Ring topology coverage. Every 2-connected simple graph admits an open-ear decomposition starting from a cycle. Consequently, root/attached cycles plus distinct-anchor ears span biconnected cyclic topology at the graph level. Spiro systems join blocks at an articulation and are represented by same-anchor cycles. Repeated composition covers fused, bridged, spiro, linked, and mixed polycycles. Chemical feasibility remains enforced per typed payload; the graph theorem does not certify strain or synthesis.

Table 7: Minimum application conditions. The implementation may impose additional grammar-specific checks.

Rule	Structural condition	Chemical/transaction condition
Atom insert	empty slot; occupied anchor	allowed atom/bond; endpoint valences feasible
Atom delete	removable atom; remainder connected	neighbor H compensation; sanitized successor
Atom restate	occupied slot	joint element/charge/H state allowed
Bond insert	absent edge; endpoints occupied	allowed order; valence feasible; nonidentity
Bond delete	existing non-bridge edge	endpoint H compensation; sanitized successor
Bond reorder	existing edge	joint order/H update feasible
Subtree Graft	old edge is a bridge; new endpoints cross its cut	atomic cut+attach; all endpoint valences feasible
Ring ear insert	valid anchors and fresh internal slots	joint typed path closes; full lowering replays
Ring ear delete	matched removable ear; remainder connected	exact inverse payload; full lowering replays
Ring restate	fixed matched cyclic system	coordinated atom/bond state passes executor

B SOURCE PRIORS AND COUPLINGS

B.1 DEGREE-BOUNDED CARBON-TREE SAMPLER

Given size law μ on $\{1, \dots, N_{\max}\}$:

1. sample $N \sim \mu$;
2. sample a Prüfer sequence with rejection or capped construction so every degree is at most four;
3. decode the tree, assign neutral carbon atom states and single bonds, and derive implicit hydrogens;
4. assert connectivity, valence validity, and the declared slot count.

The source sampler consumes only μ and an RNG seed. For a training pair, the tree draw is independent of the endpoint conditional on the selected source size.

B.2 STATIONARY SIZE JITTER

Let J be a Markov kernel on sizes with stationary law μ . Drawing $N_1 \sim \mu$ then $N_0 \sim J(\cdot | N_1)$ preserves the source marginal:

$$\Pr(N_0 = m) = \sum_n \mu(n) J(m | n) = \mu(m). \quad (13)$$

A local proposal $n \rightarrow n \pm 1$ with a Metropolis–Hastings accept/reject correction is a simple reversible choice. The jitter scale controls teacher transport cost but must not be confused with the source law. At inference, N_0 is drawn directly from μ without sampling a target size.

C PROGRAM COMPILATION ALGORITHMS

C.1 PRIMITIVE CONNECTIVITY-PRESERVING TRANSPORT

For completeness, any source can be reduced topologically to one retained atom: delete non-tree edges to expose a spanning tree, then remove leaves. Reverse the construction for a target spanning tree, add chords, and apply typed restatements. The production compiler uses direct common structure and Grafts because the one-atom route is valid but statistically wasteful.

C.2 GRAFT TRANSPORT

Given equal-order carbon tree T_0 and target G_1 :

1. align temporary source slots with target slots and choose a spanning tree $T_1 \subseteq G_1$ maximizing $|E(T_0) \cap E(T_1)|$;
2. while $T \neq T_1$, select $e \in E(T_1) \setminus E(T)$; adding e creates a unique cycle, from which select $f \in E(T) \setminus E(T_1)$; execute the corresponding atomic subtree Graft and validate;
3. restate retained atom states and tree-bond classes in a guard-feasible order;
4. install target chords with verified ring-ear or bond events;
5. canonicalize and assert exact equality with G_1 ; replay the complete program independently.

The graph-level edge-exchange loop performs at most $(n-1) - |E(T_0) \cap E(T_1)|$ iterations. Chemistry guards may reject a nominal exchange, so the implementation searches alternative cycle edges and uses a constructive fallback. A compilation failure is explicit; it is never converted to a truncated target.

C.3 RING CATALOG CONSTRUCTION

The current finite proposal catalog is mined only from training programs:

1. decompose each target into biconnected blocks and its block-cut tree;
2. compile cyclic blocks into root/attached cycles, open ears, same-anchor cycles, and electronic restatements;
3. canonicalize marks modulo slot permutation and chemically equivalent aliases;
4. count marks and retain a declared training-only vocabulary or factorized fields;
5. audit full-program coverage independently on train, validation, and test.

Frequency initializes or stratifies learning but does not define structural legality. The final factorized decoder predicts family, anchors, span, and typed atom/bond payload so that uncommon combinations need not occur as memorized fragments.

C.4 PROGRAM-CACHE REPRODUCIBILITY

Programs are stored as rewrite instructions plus byte-packed exact graph checkpoints rather than a dense graph at every progress position. A sampled state replays at most $c - 1$ validated actions from the nearest checkpoint. CPU compilation writes atomic, signature-validated shards with a manifest containing split hashes, compiler version, chemistry grammar, prior/coupling configuration, ring catalog hash, seeds, and checkpoint interval. A GPU training stage refuses to start from an incomplete cache. These choices change throughput and storage, not the mathematical path distribution.

D PROOFS

D.1 PROOF OF PROPOSITION 3

Induct on committed events. The base state is in $\mathcal{M}_\Gamma$. If $G_n \in \mathcal{M}_\Gamma$, the CTMC has support only on $a \in \mathcal{A}(G_n)$; by (3), $G_{n+1} = T_a G_n \in \mathcal{M}_\Gamma$. Local boundedness excludes explosion on the finite horizon, so only finitely many jumps occur almost surely. The conclusion applies equally to teacher programs and learned sampling because both invoke the same executor. $\square$

D.2 PROOF OF PROPOSITION 4

For connected G , choose a spanning tree T_G . Delete every edge in $E(G) \setminus E(T_G)$; each is on a cycle at deletion and is therefore not a bridge, so connectivity is preserved. Repeatedly delete leaves until one vertex remains. From one vertex, reverse leaf insertions to construct a spanning tree T_H of H , then insert edges in $E(H) \setminus E(T_H)$. Every intermediate is connected.

For aligned trees T and T_1 , take $e \in E(T_1) \setminus E(T)$. Adding e creates a unique cycle. That cycle contains an edge $f \in E(T) \setminus E(T_1)$; otherwise T_1 would contain a cycle. Replacing f by e is one subtree Graft, preserves the tree property, and increases overlap with T_1 by one. Repetition terminates after at most $(n-1) - |E(T_0) \cap E(T_1)|$ exchanges. $\square$

D.3 PROOF OF PROPOSITION 6

Fix occupied (G, t, H) and let $Y = R_t^Z(H \mid X_t, K_t)$ conditional on $X_t = G$. The part of (11) depending on scalar prediction $u \geq 0$ is

$$\ell(u) = u - \mathbb{E}[Y \mid X_t = G] \log u, \quad (14)$$

with target-only terms omitted. For $u > 0$, $\ell'(u) = 1 - \mathbb{E}[Y \mid X_t = G]/u$ and $\ell''(u) = \mathbb{E}[Y \mid X_t = G]/u^2 \geq 0$. If the mean is positive, the unique minimizer is its value; if zero, the infimum is attained at $u = 0$ by continuity. Finite sums commute with conditional expectation, so the result applies after alias aggregation. $\square$

D.4 PROOF OF COROLLARY 7

The program mixture induces marginal law p_t satisfying the Kolmogorov forward equation with Q_t from (10); this is the standard conditional-to-marginal Generator Matching identity. Under assumption 1, the finite-state forward equation has a unique solution. If $Q_{\theta,t} = Q_t$ and both laws begin at π_0 , their marginals agree for all t . By assumption 2, the program mixture terminates at p_{data} , hence so does the exact learned process. $\square$

D.5 IMPORTANCE-CORRECT PROGRESS STRATIFICATION

Let $p(k \mid t, Z)$ be the binomial progress law and $h(k \mid Z)$ a family-balanced proposal. Sample from $\tilde{p} = (1 - \rho)p + \rho h$ and weight by $w = p/\tilde{p}$. For any integrable per-progress loss g , $\mathbb{E}_{\tilde{p}}[wg] = \sum_k p(k)g(k) = \mathbb{E}_p[g]$. Since $\tilde{p} \geq (1 - \rho)p$, support is retained for $\rho < 1$.

E MODEL, TRAINING, AND SAMPLING DETAILS

E.1 NEURAL ARCHITECTURE

The graph encoder embeds atom type, formal charge, implicit hydrogen count, bond class, model time, and ring/topology context. Message passing produces node and graph representations. A graph head predicts total hazard; a masked family head predicts the rule schema; atom, pair, and template heads predict operands and typed payloads. Hierarchical normalization prevents a family with many raw matches from receiving probability solely because of multiplicity. Exact final values are: hidden width **XXX**, layers **XXX**, parameters **XXX**, batch size **XXX**, optimizer **XXX**, learning rate **XXX**, time schedule **XXX**, and training updates **XXX**.

E.2 TRAINING ESTIMATOR

For each minibatch item:

1. draw or load (G_0, G_1, Z) from the declared coupling and compiled cache;
2. sample time t and progress k , optionally from the importance-correct stratified proposal;
3. reconstruct $X_t = G_k^Z$ from its nearest packed checkpoint;
4. encode X_t once, form analytic legality masks, and evaluate (11);
5. aggregate aliases for the teacher successor and update θ .

We log hazard, family/operand accuracy, illegal-mask disagreements, gradient norm, and held-out RGM loss. Model selection uses a declared validation criterion without test FCD.

E.3 EXACT ANCESTRAL SAMPLER

Let $0 = s_0 < \dots < s_J = S$ be the operational-time grid. Starting from $G \sim \pi_0$, at grid interval j :

1. compute $\lambda = \lambda_\theta(G, s)$ and masked marked probabilities;
2. if $\lambda = 0$, advance to s_{j+1} and log a stall;
3. otherwise draw $\Delta \sim \text{Exponential}(\lambda)$;
4. if $s + \Delta \geq s_{j+1}$, set $s = s_{j+1}$ without an event;
5. else sample a hierarchically, execute $G \leftarrow T_a G$, set $s \leftarrow s + \Delta$, and recompute all rates.

The event budget is a numerical safeguard and its hit rate is reported. There is no rejection loop over invalid actions: analytic masks and the executor define zero support, and any disagreement is an auditable implementation error.

E.4 COMPLEXITY

Let $n \leq N_{\max}$, d be hidden width, L message-passing layers, and $C_r(G)$ the tensorized candidate count for family r . One rate evaluation costs the graph encoder plus masked heads, approximately $O(L(nd^2 + |E|d) + \sum_r C_r(G)d)$ for the dense implementation, rather than one graph encoding per canonical successor. Compilation is offline and program-dependent; Graft tree alignment is polynomial and each edge exchange is validated. Final measured preprocessing time, peak memory, train throughput, and sample throughput are **XXX**, **XXX**, **XXX**, and **XXX**.

F COMPLETE EXPERIMENTAL PROTOCOL

F.1 FROZEN DATA CONTRACT

We will release raw input hashes, canonicalization code, exact accepted SMILES, split indices, $N_{\max}$, element/charge/bond vocabularies, every exclusion count, and the ring-catalog audit. Test molecules do not influence the source-size law, templates, hyperparameters, or checkpoint selection.

F.2 FCD CONTRACT

FCD is computed with one frozen ChemNet implementation and environment. Invalid or null samples are not replaced. We report generated/reference counts, total FCD, additive mean and covariance terms, generated/reference covariance traces, multi-seed mean and standard deviation, and bootstrap intervals. We use at least **XXX** independent samples per final seed; any 2,000-sample diagnostic is labeled preliminary.

F.3 ABLATION PROTOCOLS

Prior/coupling. Hold architecture, data, steps, and sampler fixed; compare null, independently sized carbon tree, size-matched tree, and stationary size-jitter. Report teacher length as well as learned endpoint metrics.

Graft. Compare primitive delete/regrow transport with Graft-enabled transport on identical sampled source-target pairs. Measure compilation success, path length, one-atom visits, source-edge survival, training throughput, event use, and endpoint FCD/topology.

Rings. Compare scalar bond construction, topology-only ears, typed ears, and typed ears plus ring-system restatement. Report per-class recall/TV for aromatic, saturated, fused, bridged, spiro, small rings (3–4), ordinary rings (5–7), and macrocycles under the dataset definition.

Objective and quotient. Ablate total-hazard learning, importance-correct stratification, and canonical alias aggregation. The last ablation tests whether symmetric syntactic multiplicity biases chemical successor rates.

F.4 TRAJECTORY AUDITS

For every sampled event we record time, rule family, operands, pre/post canonical keys, hazard, and validity/connectivity flags. Aggregates include:

- rule-family and inverse-pair utilization by time and size;
- source-edge survival, topology edit distance, and one-atom-state visitation;
- ring class before and after every cyclic event;
- aliases per successor and marked-versus-quotient calibration;
- event-budget exhaustion, zero-hazard stalls, and time-grid sensitivity;
- agreement of analytic masks/direct sampling with exhaustive fibers on small graphs.

F.5 ADDITIONAL PENDING RESULTS

Table 8: Ring taxonomy distribution distances; all values pending.

Method	Aromatic	Saturated	Fused	Bridged	Spiro	Macrocycle
DiGress	XXX	XXX	XXX	XXX	XXX	XXX
Morph	XXX	XXX	XXX	XXX	XXX	XXX
COMPOSE	XXX	XXX	XXX	XXX	XXX	XXX

Table 9: Compute and storage; preprocessing is included and all values are pending.

Method	Parameters	Compile h	Train h	Peak memory	Molecules/s
COMPOSE	XXX	XXX	XXX	XXX	XXX

F.6 CONTROLLED-DESIGN AND EDITING PROTOCOL

Controlled molecular design is the paper’s primary empirical thrust (Section 6.2). We initialize from held-out CNOF leads, optionally freeze a scaffold or SMARTS motif by removing incompatible actions from $\mathcal{A}(G)$, and evaluate constraint satisfaction, generate-then-filter usable fraction by hardness, usable-oracle efficiency, constrained property gain, scaffold retention, anytime best-so-far reward, edit count, diversity, and all-step validity. The committed scaffold-retention, property-gain, and constraint-satisfaction values are **100%**, $+0.072$, and **100%**. The separate unconstrained oracle-hungry success rate (**XXX**) is reported only in this appendix with disclosed oracle budget and search-method comparators, never as a headline. Reward fine-tuning or inference guidance is reported separately from the unguided generator.

G FAILURE MODES, BROADER IMPACT, AND REPRODUCIBILITY

Failure modes. We stratify failures into compiler support, proposal-mask false negatives, executor discrepancies, zero-hazard stalls, event-budget exhaustion, source-memory artifacts, invalid ring electronics, over-fusion, macrocycle excess, and endpoint distribution mismatch. Chemical examples are selected by predeclared strata rather than only visual appeal.

Broader impact. The model can generate valid but unsafe, toxic, or controlled compounds. A distribution-learning score is not evidence of therapeutic value. Downstream property guidance requires access control, dual-use review, toxicity assessment, and wet-lab governance. Corpus bias and cheminformatics blind spots remain even when formal validity is guaranteed.

Artifact checklist. The final release will contain:

- code version, environment lockfile, and RDKit/ChemNet versions: **XXX**;

-
- dataset, split, vocabulary, and canonicalization hashes: **XXX**;
 - all hyperparameters, seeds, source draws, and coupling settings: **XXX**;
 - compiled-program manifest, ring catalog, and recovery checkpoints: **XXX**;
 - generated SMILES, per-event trajectories, and raw metric outputs: **XXX**;
 - multi-seed summaries, confidence intervals, and failed-run disclosure: **XXX**.

H LARGE LANGUAGE MODEL USAGE STATEMENT

Large language models were used for drafting and code assistance. All mathematical claims, citations, implementation details, and empirical values require author verification. No generated reference or result is retained without checking a primary source or frozen experiment artifact.